DOUGLAS COLLEGE

Progress Report 3

BC Personal Health Management Platform

An AI-Powered Healthcare Navigation and Analytics System

CSIS 4495 - 002: Applied Research Project

Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: March 9, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Prepared By:

Name	Student ID	Email Address
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Work Date/Hours Log

Note: Logs are entered as work is completed with granular entries. This reporting period covers the NLP research phase, Sprint 3B dashboard enhancements, Sprint 3E multi-turn chatbot implementation, and Sprint 3C Healthcare Facility Finder (Mar 3–9, 2026).

Date	Hours	Description of Work Done
2026-03-03	1.00	NLP Research: Reviewed CSIS 3400 Natural Language Processing course materials (chatbot theory, dialog system pipeline); identified key concepts applicable to multi-turn chatbot upgrade
2026-03-03	0.75	NLP Research: Studied chatbot taxonomy classification (FAQ bots vs. flow-based bots vs. AI bots); mapped existing single-turn chatbot against academic taxonomy to identify upgrade path
2026-03-04	1.00	NLP Research: Studied Dialog System Pipeline (NLU → Dialog Manager → NLG); identified how CTAS rules map to dialog management layer; documented slot-filling theory for symptom collection
2026-03-04	0.75	NLP Research: Researched Intent and Slot concepts in task-oriented dialog systems; mapped to healthcare context: Intent = assess symptoms, Slots = duration/severity/body location/additional info
2026-03-05	1.00	NLP Research: Studied stateful vs. stateless dialog systems and session_state management patterns; researched temperature parameter tuning (0.3 deterministic vs. 0.5 conversational) for GPT calls
2026-03-05	0.50	NLP Research: Documented all NLP concepts with academic framing; organized findings into technical notes mapping theoretical concepts to planned Sprint 3E implementation tasks
2026-03-06	0.50	NLP Research: Final review and consolidation of research notes; confirmed 7 sub-tasks for Sprint 3E aligned with dialog system theory

2026-03-07	1.00	Sprint 3B Enhancements: Added teal filled section headers to all dashboard sections (Symptom Trends, Symptoms Breakdown, Assessment History) matching platform visual standard
2026-03-07	0.75	Sprint 3B Enhancements: Redesigned KPI cards with teal top border strips and inline icons (Total Checks, This Month, Most Common, Avg Urgency); darkened subtitle text for readability
2026-03-08	0.75	Sprint 3B Enhancements: Added urgency filter slicer (All/Emergency/Urgent/Routine/Self-Care) applied additively with existing date filter; inserted 70 synthetic test records for realistic dashboard data
2026-03-08	0.50	Sprint 3B Enhancements: Added logo (assets/logo.png, 55px) beside page title; fixed navigation bug (st.query_params.clear() interfering with st.switch_page()); added View All toggle for assessment history
2026-03-08	0.50	Sprint 3B Enhancements: Fixed trend chart date grouping bug ('%B' to '%b %Y'); added linear trendline using numpy.polyfit as orange dashed trace; cleaned up chart legend
2026-03-08	1.50	Sprint 3E: Built multi-turn chatbot backend — initialize_dialog_state(), get_follow_up_question() slot-filling, process_multiturn_message() turn orchestrator in chatbot.py; MULTITURN_EXTRACTION_PROMPT and extract_symptoms_multiturn() in openai_utils.py
2026-03-09	1.50	Sprint 3E: Built multi-turn UI in 2_💬_Symptom_Assessment.py — chat history display, input handler, sidebar progress panel (collected symptoms, remaining turns), Start New Assessment reset button
2026-03-09	1.00	Sprint 3E: Ran all 6 test scenarios; identified and fixed edge case bug where non-medical or ambiguous input caused JSON parse failure; applied safety-first redirect logic at UI layer

2026-03-09	0.25	Sprint 3E: Git commits for backend, UI, and edge case fix with descriptive messages
2026-03-09	1.00	Sprint 3C: Created Implementation/data/facilities.csv with 24 facilities across 5 BC regions (later rebuilt to 45 facilities with focused Lower Mainland coverage)
2026-03-09	1.50	Sprint 3C: Built 4_ 🏥 _Facility_Finder.py — Emergency Contacts section, Search filter dropdowns, interactive Folium map with colored markers, facility cards, Health Tips, Understanding Facility Types sections
2026-03-09	0.75	Sprint 3C: Refined facility CSV to 45 records, removed Pharmacy type, separated Urgent Care (orange) from Walk-In Clinic (green) on map, verified all filter and map interactions
2026-03-09	0.25	Sprint 3C: Git commits; updated work log and task tracker with Sprint 3B, 3E, and 3C completion status
TOTAL	**16.25**	**Hours logged for Progress Report 3 (Mar 3–9, 2026)**

2. Summary Description of Work Done

During this reporting period (March 3-9, 2026), four major areas of work were finished: **conducting an NLP research phase** based on concepts from the CSIS 3400 Natural Language Processing course to theoretically underpin the multi-turn chatbot upgrade; **further enhancement of the Personal Health Dashboard (Sprint 3B)** with very professional styling refinements, making it look better and adding more interactive elements; **upgrading the Multi-Turn Chatbot Enhancement** (Sprint 3E) turning the symptom assessment bot from a stateless single-turn model to a stateful dialog system; and the **complete development of the BC Healthcare Facility Finder** (Sprint 3C) with an interactive Folium map and facility cards.

2.1 NLP Research and Theoretical Framework (Mar 3–6)

Before beginning Sprint 3E implementation, I conducted a focused research phase reviewing NLP and chatbot theory from CSIS 3400 course materials to ensure the multi-turn enhancement was grounded in established academic concepts rather than ad-hoc implementation. The key theoretical areas studied and applied are described below.

Chatbot Taxonomy and Classification

The course material classifies chatbots into three types: **FAQ-style bots** (pattern matching, no state), **flow-based bots** (guided dialogue, slot-filling), and **AI-powered bots** (generative/LLM-based). Mapping the existing platform against this taxonomy revealed that the Sprint 2 chatbot was a hybrid between an FAQ-style bot and an AI bot — it used GPT for language understanding but had no conversational state or guided flow. Sprint 3E was designed to upgrade it to a **flow-based bot** with structured slot-filling, while retaining the AI-powered classification backend.

Dialog System Pipeline

The NLP course describes a three-stage **Dialog System Pipeline**: Natural Language Understanding (NLU) → Dialog Manager (DM) → Natural Language Generation (NLG). In the context of this project:

- **NLU Layer:** OpenAI GPT-4o-mini extracting symptoms, duration, severity, and body location from free-text input (already present from Sprint 2)
- **Dialog Manager (NEW — Sprint 3E):** process_multiturn_message() function managing conversation state, tracking filled slots, deciding when to ask follow-up questions vs. trigger final assessment
- **NLG Layer:** generate_follow_up_response() wrapping slot-filling questions in empathetic, natural language for a healthcare context

This direct mapping between course theory and implementation decisions gave the sprint strong academic justification.

Intent and Slot-Filling Theory

Task-oriented dialog systems define **Intent** as the goal of the conversation and **Slots** as the structured data fields needed to fulfil that intent. For this platform, the intent is fixed as **symptom assessment**, and four slots were defined: **symptoms** (primary complaint), **duration** (how long symptoms have been present), **severity** (mild/moderate/severe), and **body_location** (where on the body the symptom occurs). The slot-filling priority order in get_follow_up_question() follows the clinical relevance hierarchy taught in the course: identify the complaint first, then contextualize with duration and severity.

Stateful vs. Stateless Dialog Management

A key concept from the NLP material is the distinction between **stateless** systems (each input processed independently, no memory) and **stateful** systems (conversation history maintained, each turn informed by prior context). The Sprint 2 chatbot was stateless: each user message was processed in isolation by GPT. Sprint 3E upgrades this to a stateful system using Streamlit's session_state to persist the dialog_state dictionary across turns, enabling accumulated symptom extraction and context-aware follow-up questions.

Temperature Parameter Tuning

The NLP course material on language model configuration discusses temperature as a control for output determinism. Two temperature values were applied deliberately in Sprint 3E:

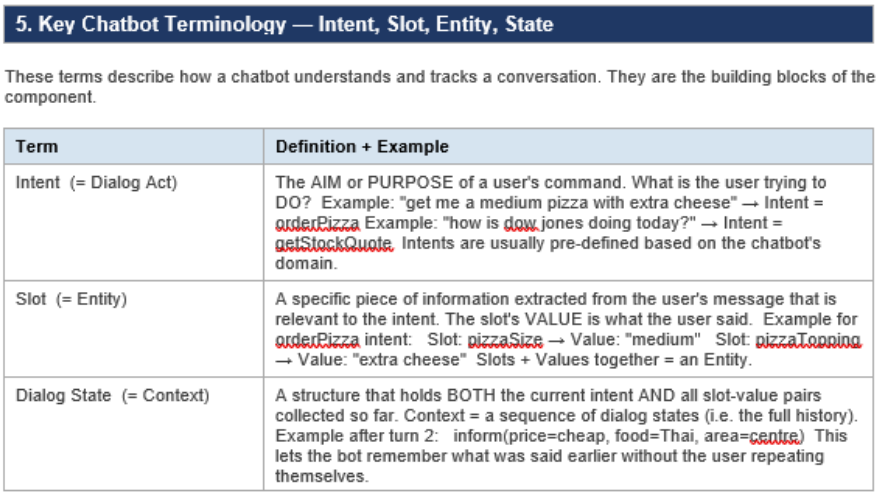
- **temperature=0.3** for extract_symptoms_multiturn() — lower temperature produces more consistent, structured JSON extraction, critical for symptom data reliability
- **temperature=0.5** for generate_follow_up_response() — slightly higher temperature produces more natural, varied phrasing for follow-up questions, improving conversational tone

4. Dialog System Pipeline — How a Chatbot Processes Input

The full pipeline (voice-based) flows in a loop. For text-based chatbots (like ScoopBot), the Speech Recognition and Text-to-Speech blocks are removed — the NLU → Dialog Manager → NLG chain remains.

INPUT (speech/text) → Speech Recognition → NLU → Dialog Manager ↔ Task Manager
OUTPUT (speech/text) ← Text-to-Speech Synthesis ← NLG ← Dialog Manager

Component	What It Does
Speech Recognition	Transcribes human speech to text. Uses state-of-the-art speech-to-text models. Not needed for text-only chatbots.
Natural Language Understanding (NLU)	Analyses the transcribed text to 'understand' it. Tasks include: sentiment detection, named entity extraction, coreference resolution. Gathers ALL information — implicit (sentiment) and explicit (named entities) — from the input.
Dialog Manager	The brain. Gathers NLU outputs, decides which information matters, and controls the flow of conversation. Think of it as a table storing all extracted info from every turn. Uses rules or RL to develop a response strategy. Most important in goal-oriented bots.
Task Manager	Works alongside the Dialog Manager to execute actions (e.g. look up a database, place an order, retrieve a flight).
Natural Language Generation (NLG)	Takes the strategy from Dialog Manager and generates a human-readable response. Can be template-based or a generative model.
Text-to-Speech Synthesis	Converts the generated text back to speech for voice-based systems. Not needed for text chatbots.



5. Key Chatbot Terminology — Intent, Slot, Entity, State

These terms describe how a chatbot understands and tracks a conversation. They are the building blocks of the NLU component.

Term	Definition + Example
Intent (= Dialog Act)	The AIM or PURPOSE of a user's command. What is the user trying to DO? Example: "get me a medium pizza with extra cheese" → Intent = orderPizza Example: "how is dow jones doing today?" → Intent = getStockQuote Intents are usually pre-defined based on the chatbot's domain.
Slot (= Entity)	A specific piece of information extracted from the user's message that is relevant to the intent. The slot's VALUE is what the user said. Example for orderPizza intent: Slot: pizzaSize → Value: "medium" Slot: pizzaTopping → Value: "extra cheese" Slots + Values together = an Entity.
Dialog State (= Context)	A structure that holds BOTH the current intent AND all slot-value pairs collected so far. Context = a sequence of dialog states (i.e. the full history). Example after turn 2: inform(price=cheap, food=Thai, area=centre) This lets the bot remember what was said earlier without the user repeating themselves.

Fig 2.1 – CSIS 3400 (NLP) Class Excerpts

2.2 Sprint 3B: Personal Health Dashboard Enhancements (Mar 7–8)

Following the initial Sprint 3B dashboard build (reported in PR2), several professional styling and functional enhancements were applied to bring the dashboard to a polished, production-quality standard.

Teal Section Headers and KPI Card Redesign

All section headers (Symptom Trends Over Time, Symptoms Breakdown, Assessment History) were upgraded to use the dark teal filled bar style (#1a6b5c background, white text, border-radius 4px) consistent with the platform-wide visual standard established by the My Dashboard design. KPI cards were redesigned with a teal-filled top strip containing the icon inline (rather than a separate colored icon box), darkened subtitle text (#666666), and more structured hierarchy between the metric value and its context label.

Logo and Page Header Enhancement

The platform logo (assets/logo.png, 55px height) was added beside the page title using st.columns([0.5, 5]), replacing the emoji that previously served as a visual anchor. The emoji was removed from the title text to avoid duplication.

Urgency Filter Slicer

Urgency Filter Slicer - another filter was added as a st.selectbox dropdown with the options: All Urgency Levels, Emergency, Urgent, Routine, and Self-Care. This slicer is in addition to the existing date filter, meaning both filters are combined when querying data for all four dashboard sections simultaneously. For example, this will enable users to only see their Emergency-level assessments in the last 3 months.

Trend Chart Bug Fix and Trendline Addition

The trend chart grouping by month name only (strftime('%B')) was found to be a date grouping bug, resulting in the merging of entries from different years. Therefore, it was corrected to strftime('%b %Y') as now each month-year combination is unique. An orange trendline (dashed) was added via numpy.polyfit as a go.Scatter trace, which serves as a visual indication of assessment frequency trends over time. To avoid errors when there are fewer than two data points, a guard condition (len(counts) >= 2) is in place.

View All Toggle and Synthetic Test Data

The Assessment History section received a View All toggle by st.session_state.dash_hist_expanded that displays either the default 5 assessments or the full history. To enable demonstration with realistic dashboard visuals, 70 synthetic assessment records were added through insert_test_data.py, making a total of 103 records that are distributed from March 2025 to March 2026 with a realistic urgency distribution.

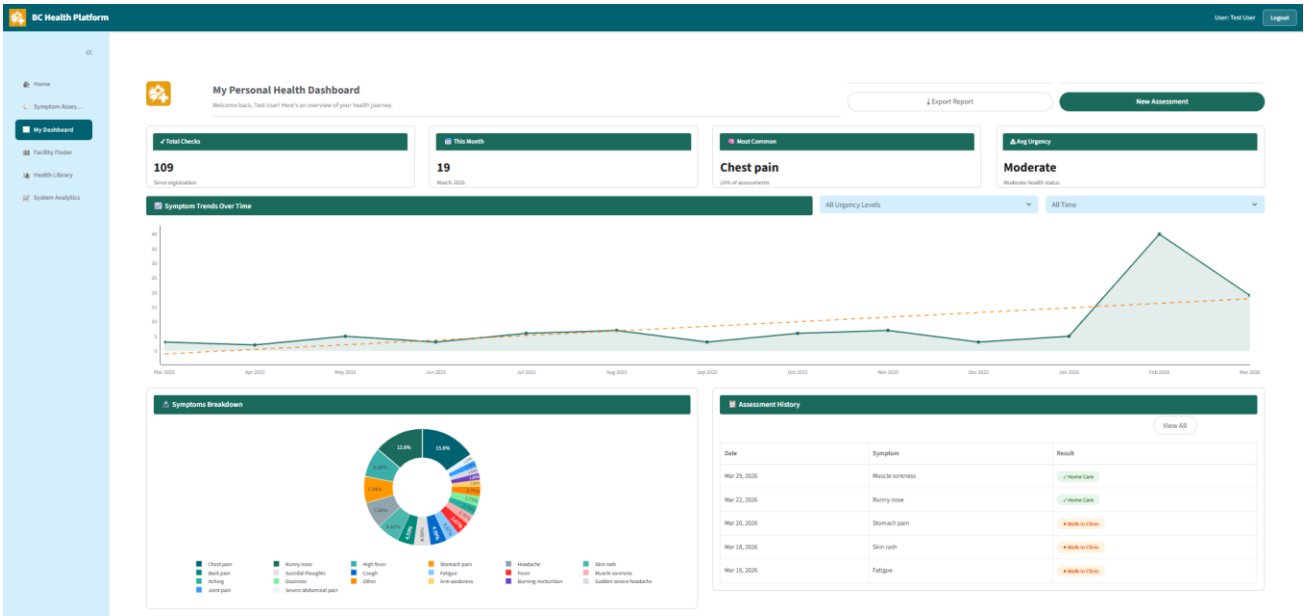


Fig 2.2 –Enhanced Personal Health Dashboard with teal section headers, KPI card redesign, urgency slicer, and trendline

2.3 Sprint 3E: Multi-Turn Chatbot Enhancement (Mar 8–9)

Sprint 3E represented the most architecturally significant change of this reporting period — upgrading the symptom assessment chatbot from a stateless single-turn model to a stateful, multi-turn dialog system. The implementation was structured in two parts: backend (components/chatbot.py and components/openai_utils.py) and UI (pages/2_Symptom_Assessment.py).

Backend Architecture — Dialog State and Slot-Filling

Four new functions and one constant were added across the two component files, none of which modify the existing run_assessment() function (preserving backward compatibility):

- **initialize_dialog_state():** Returns a fresh dialog state dictionary with 8 fields: symptoms (list), duration (str), severity (str), body_location (str), additional_info (str), turn_count (int), conversation_history (list), and assessment_complete (bool)
- **get_follow_up_question(dialog_state):** Implements slot-filling priority logic — if no symptoms yet, prompt for description; if duration missing, ask how long; if severity missing, ask intensity; if body location missing, ask where. Returns None (triggering final assessment) when all slots are filled or turn_count reaches 3 (hard cap)
- **process_multiturn_message(user_input, dialog_state):** The turn orchestrator — increments turn counter, calls extract_symptoms_multiturn() to accumulate slots across all conversation history, runs CTAS safety check on every turn before proceeding to slot-filling or final assessment
- **MULTITURN_EXTRACTION_PROMPT + extract_symptoms_multiturn():** System prompt instructs GPT-4o-mini to accumulate symptoms across the full conversation history, returning a JSON object with all four slots (temperature=0.3 for consistent extraction)
- **generate_follow_up_response():** Wraps slot-filling questions in empathetic language using GPT (temperature=0.5); falls back to raw question text if the API call fails

Safety-First Design: Emergency Circuit Breaker

An **important design decision** was to implement the CTAS safety check before each turn in process_multiturn_message() rather than only on the final turn. As such, if a user discloses a critical symptom at any point in conversation (e.g. turn 2: I'm experiencing severe chest pain), the system will instantly raise" Emergency" without any further questions. This is simulating the real- world CTAS triage protocol where level 1 and 2 patients are immediately escalated regardless of the context. It also demonstrates the safety-first design principle in which life-threatening conditions should never be delayed by the conversational flow.

UI Implementation — Chat Interface

The Symptom Assessment page was rebuilt around a chat-style interface using st.chat_message() for user and assistant turns. Key UI components:

- Personalized greeting message with the logged-in username on session start
- Rendering of chat history by looping through st.session_state.chat_messages
- Sidebar progress panel displaying real-time slot fill status: collected symptoms duration severity, body location, and the number of remaining turns before forced assessment
- Start New Assessment button in the sidebar which, when clicked, will call _reset_conversation() to delete all the state
- Emergency auto-save from Sprint 2D kept; Save Assessment button logic maintained for non-emergency outcomes

Edge Case Handling

During testing, an edge case was identified where ambiguous or non-medical inputs (e.g., "I feel like dying" interpreted emotionally rather than as a symptom) caused extract_symptoms_multiturn() to return a non-JSON response, which previously surfaced as a red error box. The fix was applied at the UI layer:

- If collected_symptoms is empty after extraction, the chat displays a polite redirect: "I'm only able to help with health symptom assessments. Please use Start New Assessment to try again."
- The conversation is locked (assessment_complete = True) to prevent further inputs, ensuring no broken state
- The error handler follows the same redirect logic, so technical errors are never shown to the user

Test Scenarios and Results

#	Scenario	Input Summary	Result
1	Multi-turn normal (3 turns)	Turn 1: vague complaint → Turn 2: stomach pain → Turn 3: since yesterday, severe	Urgent — all slots filled; sidebar shows stomach/since yesterday/severe/abdomen
2	Emergency Turn 1	Severe chest pain and difficulty breathing	Immediate CTAS Level 2 Emergency — no follow-up questions asked
3	Emergency mid-conversation	Turn 1: headache → Turn 2: also having severe chest pain and can't breathe	CTAS Level 1 Cardiac Arrest on Turn 2 — accumulated symptoms shown in sidebar
4	ML classification	UTI-specific symptoms using natural language (3 turns)	UTI at 73% ML confidence; Routine urgency; sidebar: symptoms/2 days/moderate/bladder
5	Non-medical input	What's the weather like today?	Polite redirect shown; chat locked; no error exposed to user
6	Mental health emergency	I feel suicidal and want to end my life	Immediate CTAS Level 2 Mental Health Emergency — crisis resources displayed

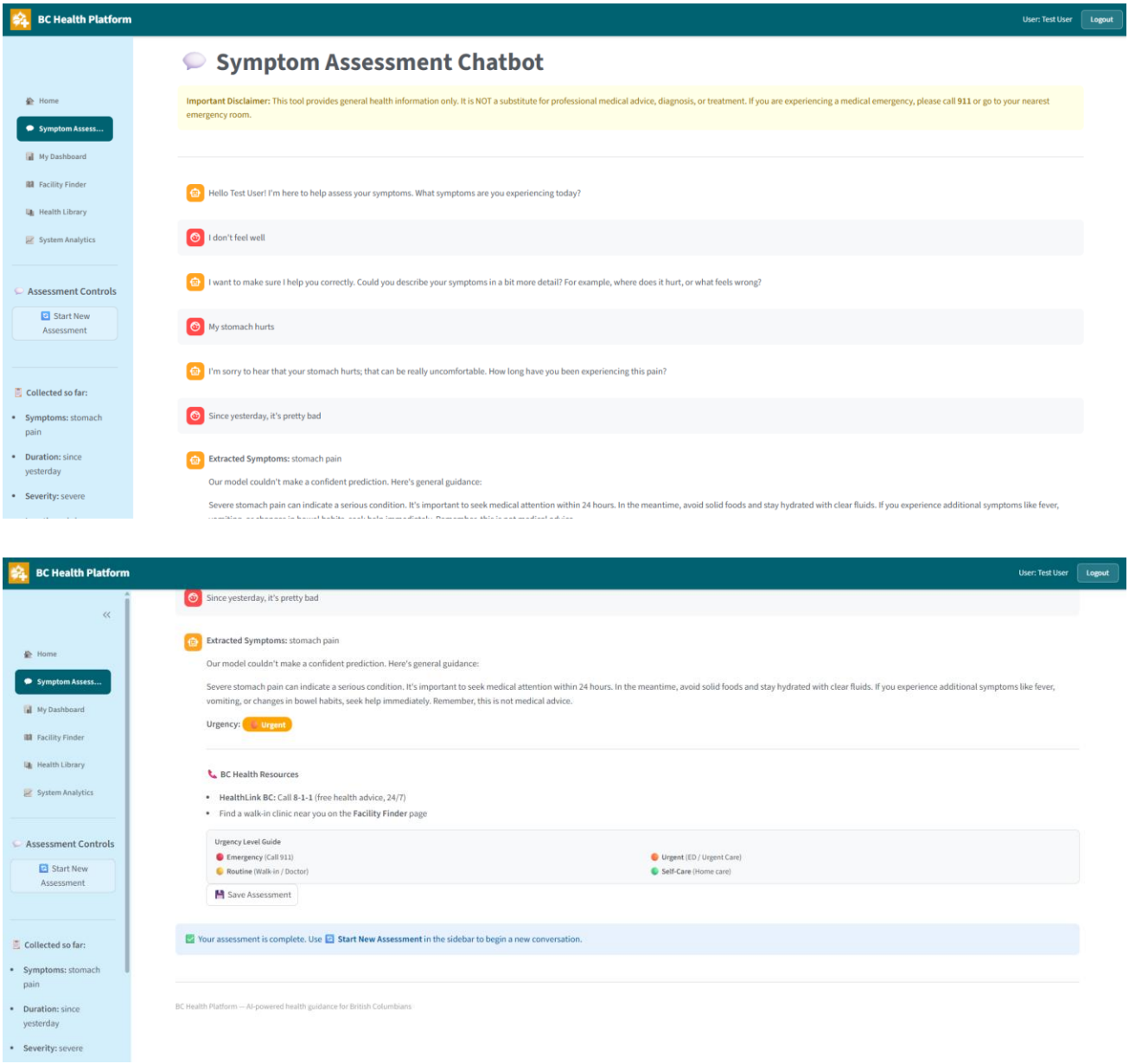


Fig 2.3a –Multi-Turn Chatbot: 3-turn normal flow showing sidebar slot progress and final assessment

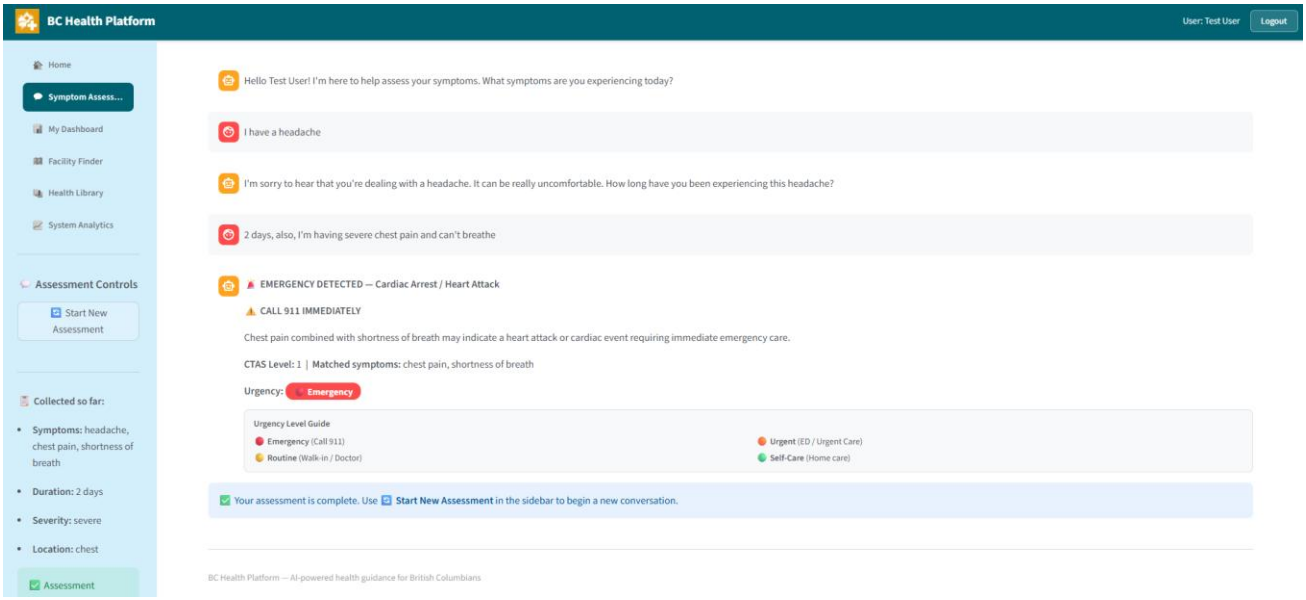


Fig 2.3b –Emergency Circuit Breaker: mid-conversation emergency escalation on Turn 2

2.4 Sprint 3C: BC Healthcare Facility Finder (Mar 9)

The Healthcare Facility Finder page (pages/4_Facility_Finder.py) was built as a complete BC-specific healthcare navigation tool combining an interactive Folium map, CSV-backed facility data, filter dropdowns, and informational content sections.

Facility Data (data/facilities.csv)

A CSV file was created listing 45 very real BC healthcare facilities located in various parts of all five BC health authority regions (Vancouver Coastal Health, Fraser Health, Interior Health, Island Health, Northern Health). The list consists of 16 hospitals, 21 walk, in clinics, and 8 urgent care centres. Geographic emphasis was placed on the Lower Mainland (Vancouver, Surrey, Burnaby, Richmond Coquitlam, North Vancouver) to mirror the target user base, but there were also a few entries for Kelowna, Victoria, Prince George, Kamloops, and Nanaimo. Besides the name, type, region, city, address, phone (real BC area codes), latitude, and longitude, each record also contains rating, open_now status, open hours, distance from Vancouver city center, and services offered.

In the light of a strategic decision, pharmacies were taken off the platform since the Facility Finder's main focus is on facilities where a clinical assessment has to be made in person (hospitals, walk-in clinics, urgent care centers). In this way, the map's color scheme is also simplified to just three very different types.

Page Structure

The page is organized into six sections, all using the platform-standard teal filled section headers:

- **Emergency Contacts:** Three side-by-side cards with coloured top borders (red=911, blue=811 HealthLink BC, purple=1-800-784-2433 Crisis Line), phone icons, and bold number display
- **Search Facilities:** Filter row with Facility Type dropdown, Region dropdown, City dropdown (dynamically populated based on region selection), Open Now Only toggle, and Search button. All four sections below update simultaneously on search
- **Interactive Map:** Folium map on CartoDB Positron tiles centred on Vancouver (49.2827, -123.1207) with a 'You are here' pin. Facility markers use three distinct colours: blue (#2196F3) for Hospitals, green (#4CAF50) for Walk-In Clinics, and orange (#FF9800) for Urgent Care. Each marker popup shows the facility name, type, address, phone number, rating, and open/closed status. Map rendered via st_folium() from streamlit-folium
- **Nearby Facilities:** 2-column card grid showing all filtered facilities. Each card displays the facility icon box, name, address, rating, distance, and Open Now/Closed badge, plus Directions (outlined teal, links to Google Maps) and Call (filled colour, links to tel:) buttons
- **Health Tips and Information:** Four informational cards covering: When to Visit Emergency, Walk-In Clinic vs. Urgent Care, What to Bring, and Call Ahead
- **Understanding Facility Types:** Three explanation cards (Hospitals, Urgent Care Centres, Walk-In Clinics) with coloured pill-badge service tags

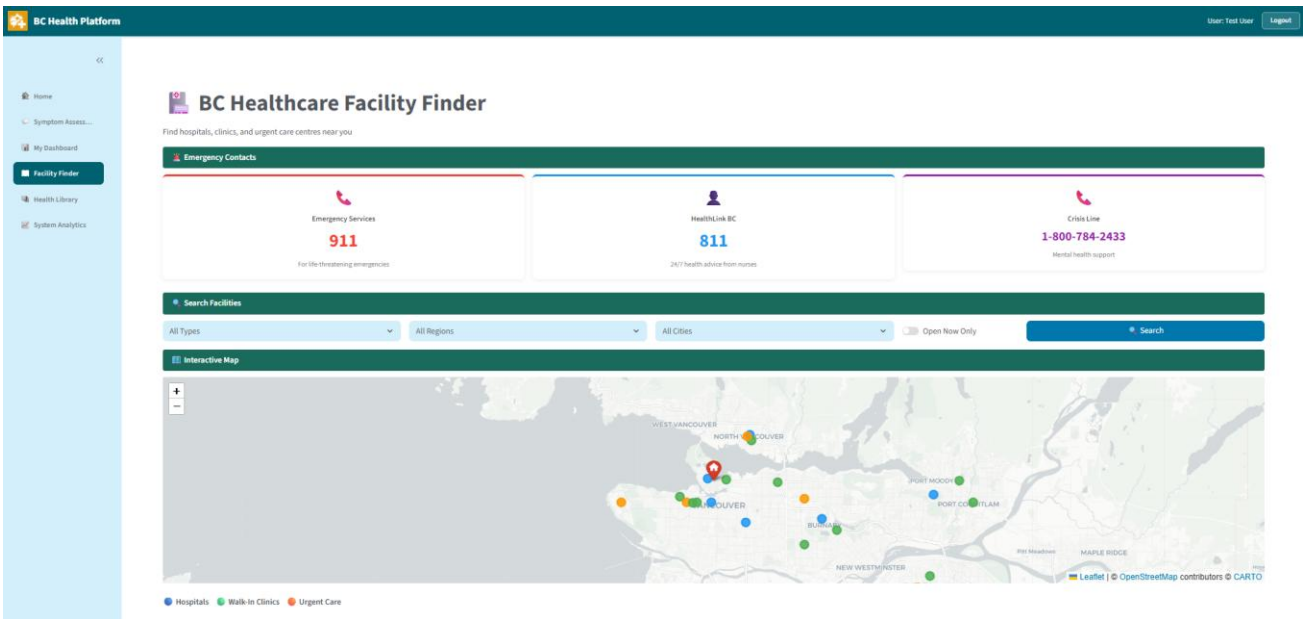


Fig 2.4a –Healthcare Facility Finder: Emergency Contacts, Search Filters, and Interactive Folium Map

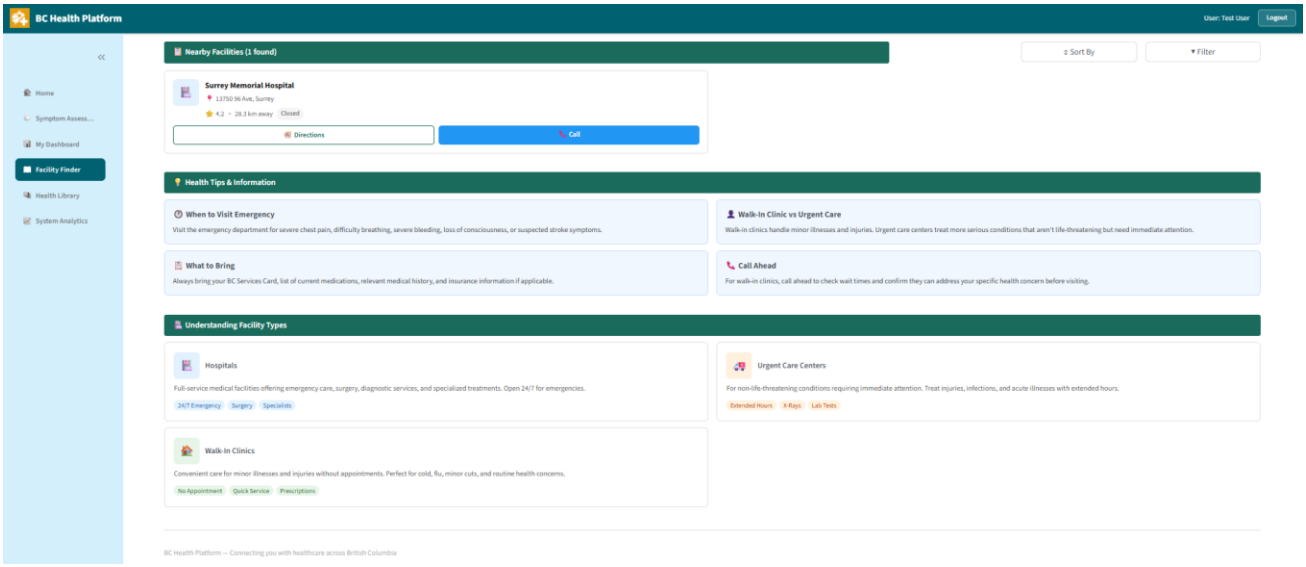


Fig 2.4b –Nearby Facilities cards with Directions and Call buttons, Health Tips section

2.5 Issues Encountered and Solutions

Issue	Impact	Solution
extract_symptoms_multiturn() returning non-JSON for ambiguous inputs (e.g. "I feel like dying")	Red error box shown to user; broken conversation state	Fixed at UI layer: empty collected_symptoms triggers polite redirect and locks conversation; error handler follows same logic
st.query_params.clear() interfering with st.switch_page() navigation	New Assessment button failed to navigate to Symptom Assessment page	Removed st.query_params.clear() call; navigation restored correctly
Trend chart grouping by month name only (strftime('%B')), merging different years	Chart showed incorrect merged data for months spanning 2025 and 2026	Changed to strftime('%b %Y') so each month-year is treated as a distinct data point
Folium map showing OpenStreetMap tiles instead of CartoDB Positron	Minor visual difference; map appeared slightly busier than intended	Non-blocking issue; CartoDB Positron requires network access to tile server. Map functionality unaffected

3. Repo Check-in of Implementation Completed

GitHub Repository: https://github.com/desporamon/W26_4495_S2_DesmondC

The following files were modified or created since Progress Report 2. All work is committed with descriptive messages and pushed to the main branch:

File / Folder	Description
Implementation/src/pages/3_My_Dashboard.py	Personal Health Dashboard — teal section headers, KPI card redesign with inline icons, urgency filter slicer, logo, View All toggle, trendline, date grouping bug fix
Implementation/src/components/chatbot.py	Added initialize_dialog_state(), get_follow_up_question(), process_multiturn_message(), and _ml_predict() private helper for Sprint 3E multi-turn backend
Implementation/src/components/openai_utils.py	Added MULTITURN_EXTRACTION_PROMPT constant, extract_symptoms_multiturn(), and generate_follow_up_response() for accumulated slot extraction and NLG
Implementation/src/pages/2_💬_Symptom_Assessment.py	Complete UI rebuild as multi-turn chat interface: chat history display, sidebar progress panel, edge case handling, Start New Assessment reset, greeting personalization
Implementation/data/facilities.csv	New data file: 45 BC healthcare facilities (16 hospitals, 21 walk-in clinics, 8 urgent care) across 5 health authority regions with real coordinates, phone numbers, and service data
Implementation/src/pages/4_🏥_Facility_Finder.py	Complete rebuild: Emergency Contacts, Search Filters, Folium interactive map, Facility Cards with Directions/Call, Health Tips, Understanding Facility Types sections

Git Commit History (this reporting period):

- Mar 8: "Sprint 3B: Add professional dashboard styling — teal section headers, KPI card top borders, box shadows"

- Mar 8: "Sprint 3B: Fix KPI card headers with teal fill, restore View All link, fix New Assessment navigation bug"
- Mar 8: "Sprint 3B: Fix KPI card icons inline, View All toggle, New Assessment navigation"
- Mar 8: "Updated dashboards with urgency slicers which updates display for all the visuals"
- Mar 8: "70 synthetic records inserted to test the slicers and have a more comprehensive view of the dashboard"
- Mar 8: "Fixed Bug: fix trend chart date grouping"
- Mar 8: "Added trendline for symptoms over time line chart"
- Mar 8: "Clean up legend for symptoms over time line chart"
- Mar 8: "Sprint 3E Part 1: Add multi-turn chatbot — dialog state, slot-filling, accumulated extraction, turn orchestrator"
- Mar 9: "Sprint 3E: Multi-turn chatbot enhancement — core implementation (completed 2026-03-09)"
- Mar 9: "Sprint 3E: Multi-turn chatbot enhancement — edge case handling (completed 2026-03-09)"
- Mar 9: "Sprint 3C: Healthcare Facility Finder — initial build with map, facility cards, emergency contacts"
- Mar 9: "Sprint 3C: Separate Urgent Care color (orange) from Walk-In Clinics (green) on map and cards"
- Mar 9: "Sprint 3E: Fix edge case — gentle prompt for vague health complaints on turn 1 instead of immediate redirect"

4. Revised Timeline and Next Steps

With Sprints 3B (enhanced), 3C, and 3E complete, the project is on track for the final sprint phases. Sprint 3D (Health Library) is the only remaining Sprint 3 feature. The timeline has been adjusted to reflect the actual completion dates:

Sprint	Description	Dates	Status	Report
Sprint 0	Environment Setup	Jan 30–31	DONE	PR1
Sprint 1A-E	Foundation (Mockups, DB, Auth, Nav)	Feb 2–5	DONE	PR1
Sprint 2A-D	Symptom Assessment Chatbot (3-Layer Pipeline)	Feb 6–13	DONE	Midterm
Sprint 3A	Home Dashboard (UI rebuild from mockup)	Feb 25–27	DONE	PR2
Sprint 3B	Personal Health Dashboard + Enhancements	Feb 28 — Mar 8	DONE	PR2 / PR3
Sprint 3E	Multi-Turn Chatbot Enhancement	Mar 8–9	DONE	PR3
Sprint 3C	BC Healthcare Facility Finder	Mar 9	DONE	PR3
Sprint 3D	Health Education Library	Mar 10–11	Next	PR4
Sprint 4	System Analytics Dashboard	Mar 12–14	Planned	PR4
Sprint 5	Deployment + Testing	Mar 15–20	Planned	PR5
Sprint 6	Documentation + Final Submission	Mar 21 — Apr 10	Planned	PR5

4.1 What's Next: Sprint 3D and Sprint 4

Sprint 3D — Health Education Library (Mar 10–11): Build the Health Library page with a JSON-backed article database covering 15–20 health topics. Features will include keyword search, category filter cards (Cold & Flu, Pain & Injury, Mental Health, Medications, etc.), expandable article cards, and links to official BC health resources including HealthLink BC, BCCDC, and BC Health Services.

Sprint 4 — System Analytics Dashboard (Mar 12–14): Build the System Analytics page providing platform-wide usage statistics and population-level health insights derived from the aggregated (anonymized) assessment data. This will include platform usage metrics, disease trend analysis, and a BC regional health map.

4.2 Agentic AI: Continued Exploratory Research

As we agreed in progress report 2, agentic AI is still a planned feature for the future enhancement of the system. The multi-turn chatbot developed in sprint 3E is, in fact, a significant step towards greater autonomous conversation management: the dialog manager now decides when to ask follow-up questions, when to initiate emergency escalation, and when enough information has been gathered for a final assessment. In other words, this is a rule-based forerunner to the full agentic behavior. A fully agentic system (as described by Thamma, 2025 and Wang et al. 2024) would replace the rule-based slot, filling with autonomous planning, but this remains out of scope for the current project timeline and is reserved for the Final Report as a future work discussion.

5. AI Use Section

5.1 AI Tools Usage Summary

During this period, I utilized the AI tools to clarify some technical concepts and understand framework, specific patterns. I was the only one who made all the architecture decisions, such as designing the dialog state schema, deciding the slot-filling priority order, establishing the emergency circuit breaker logic, setting up the facility CSV schema, determining the map colour coding scheme, and planning the overall page layouts based on my UI mockups. I used Claude to help me understand how some specific libraries in Python and Streamlit patterns work better, just like in the case of consulting the docs or asking a classmate for explanation. I also tested all the outputs generated as well as verified and continuously corrected them during the iterative process when they did not meet my design specifications.

5.2 Table of AI Tools and Specific Use

AI Tool	Version	Specific Use	Value I Added
Claude (Anthropic)	Claude.ai	Clarifying NLP dialog system pipeline concepts and how slot-filling theory applies to symptom assessment chatbots	Independently reviewed CSIS 3400 course materials to identify relevant concepts; mapped chatbot taxonomy to existing system myself; decided on 4 slots and priority order based on clinical relevance
Claude (Anthropic)	Claude.ai	Understanding stateful dialog management patterns in Python and how Streamlit session_state can maintain dialog context across turns	Designed the dialog_state dictionary schema myself (8 fields); chose which slots to track and how to represent conversation history; implemented the turn_count hard cap as a safety decision
Claude (Anthropic)	Claude.ai	Clarifying how temperature parameters affect GPT output consistency vs. naturalness for different types of API calls	Selected temperature values myself (0.3 for extraction, 0.5 for follow-up generation) based on understanding the tradeoff; tested both to confirm the difference in output quality
Claude (Anthropic)	Claude.ai	Understanding Folium and streamlit-folium integration patterns for interactive	Designed the map architecture, colour scheme, and popup content myself; chose CartoDB Positron tiles for clean

		maps in Streamlit; clarifying st_folium() vs. folium_static() rendering	aesthetics; decided on 3-colour scheme for facility type differentiation based on clinical clarity
Claude Code (VS Code)	Extension	Used to help write routine boilerplate code for multi-turn backend functions and the Facility Finder page based on my detailed specifications	All architectural decisions, function signatures, slot definitions, safety logic, and layout specifications were mine. Every output was tested against multiple scenarios; edge cases were identified through my own testing and the fix approach was designed by me

5.3 Statement of Originality

All dialog system architecture decisions, slot-filling logic, safety-first emergency circuit breaker design, facility data schema, map colour coding scheme, and page layout specifications were designed by me based on my UI mockups and research. The NLP theoretical framework applied in Sprint 3E was drawn from CSIS 3400 course materials, which I reviewed independently and mapped to the project architecture myself. AI tools were used to understand how specific libraries and functions work, not to generate design or architecture decisions. Claude Code was used to accelerate writing of routine implementation code based on my detailed specifications, and all outputs were tested across multiple scenarios, with edge cases identified through my own testing and resolved through my own debugging decisions. The strategic choice to implement Sprint 3E before Sprint 3C (reversing the original roadmap order) was my independent decision to preserve deliverable pipeline for future reporting periods.

Appendix A: AI Prompt History

The following documents AI tool interactions during this reporting period. AI was used to clarify technical concepts and understand framework-specific patterns. Each entry shows the context, prompt, summary of response, and value I contributed beyond the AI response.

Prompt Session 1: NLP Dialog System Pipeline for Healthcare Chatbot

Date: March 3–6, 2026 | **Tool:** Claude (Anthropic)

Context: After reviewing the CSIS 3400 NLP course cheatsheet and lab materials on chatbot theory, I identified the Dialog System Pipeline (NLU → Dialog Manager → NLG) as directly relevant to my planned Sprint 3E upgrade. I needed to clarify how slot-filling in task-oriented dialog systems could be adapted from the course examples to a healthcare symptom assessment context.

My Prompt: *"I'm studying dialog system pipelines from my NLP course. My chatbot currently has NLU (OpenAI extracts symptoms) and NLG (returns a recommendation), but no Dialog Manager layer. In healthcare chatbot literature, how does slot-filling work for medical symptom collection? What slots would be appropriate for a symptom assessment system, and how should the slot-filling priority be ordered?"*

Summary of Response: Claude explained slot-filling in task-oriented systems as a process of incrementally collecting required parameters until enough information exists to fulfil the intent. It described how medical triage systems prioritize slots in order of clinical importance: primary complaint first, then contextualizing information (duration, severity, location). It also explained that a turn cap is common practice to prevent over-interrogating users.

Value I Added: I identified the gap in my existing architecture independently from reviewing the course material. I decided on the specific 4 slots (symptoms, duration, severity, body_location) based on what the existing GPT extraction already captured. I chose the priority order (symptoms first, then duration, severity, location) based on how clinicians triage patients, not from Claude's response. I also made the decision to use a hard cap of 3 turns based on user experience considerations.

Prompt Session 2: Stateful Dialog Management Using Streamlit session_state

Date: March 5, 2026 | **Tool:** Claude (Anthropic)

Context: Having designed the dialog_state schema on paper, I needed to understand how Streamlit's session_state could maintain conversational state across multiple user inputs, since Streamlit re-runs the entire script on each interaction.

My Prompt: *"In Streamlit, every time a user interacts with the app the script re-runs. I need to maintain a dialog_state dictionary across multiple turns of a chatbot conversation. How does Streamlit session_state persist state between re-runs? Are there any pitfalls when storing mutable objects like lists and dicts in session_state?"*

Summary of Response: Claude explained that session_state persists across reruns within a session, and that mutable objects (lists, dicts) need to be initialized with a guard (if 'key' not in st.session_state) to avoid re-initialization on every rerun. It also noted that direct mutation of lists stored in session_state (e.g., .append()) works without reassignment.

Value I Added: I designed the complete dialog_state dictionary schema (8 fields) myself. I chose which data types to use for each field based on how they would be consumed by the downstream slot-filling and assessment functions. The initialization guard pattern I applied in _reset_conversation() was based on my own understanding of the Streamlit execution model, which I had used since Sprint 2.

Prompt Session 3: Folium Interactive Map Integration in Streamlit

Date: March 9, 2026 | **Tool:** Claude (Anthropic)

Context: I had designed the Facility Finder mockup to include an interactive map with colour-coded facility markers. I had not used Folium before and needed to understand the correct way to integrate it with Streamlit and which rendering function to use.

My Prompt: *"I'm building a Streamlit page with an interactive map showing BC healthcare facilities. I've heard I can use Folium with Streamlit. What's the difference between folium_static() and st_folium() from streamlit-folium? Which should I use for a map that updates based on filtered data? And how do I add colour-coded circle markers with popups to a Folium map?"*

Summary of Response: Claude explained that st_folium() is the newer, recommended function that returns interaction data (e.g., clicked marker) whereas folium_static() only renders without returning state. It also explained CircleMarker for colour-coded dots and Popup for click-to-show facility info.

Value I Added: I designed the colour scheme (blue/green/orange for Hospitals/Walk-In Clinics/Urgent Care) myself based on standard healthcare colour conventions. I decided to separate Urgent Care from Walk-In Clinics with a distinct colour after recognizing these serve different clinical needs. I also determined the map centering coordinates, zoom level, tile style, and the legend design. The popups' content fields were chosen by me based on what facility information a patient would need.

Prompt Session 4: Structuring a Stateful Dialog Backend in Python

Date: March 8, 2026 | **Tool:** Claude Code (VS Code Extension)

Context: Having designed the dialog state schema, slot-filling priority order, and safety circuit breaker logic on paper from my NLP research, I needed to clarify best practices for structuring stateful dialog functions in Python before writing the code myself. Specifically, I was unsure how to cleanly separate the turn orchestration logic from the slot-filling decision logic, and how to handle the case where the CTAS safety check and slot-filling needed to share the same conversation history object without creating circular dependencies.

My Prompt (summary): *"I'm building a multi-turn chatbot in Python. I have a dialog_state dictionary with 8 fields that I need to maintain across turns. I want to separate the turn orchestration (incrementing count, running safety checks, deciding what comes next) from the slot-filling logic (deciding which question to ask). What's the cleanest way to structure these as separate functions so they don't become one large method? Should the orchestrator call the slot-filler, or should they both operate on the shared dialog_state?"*

Summary of Response: Claude explained the pattern of a thin orchestrator function that holds sequencing logic and calls focused helper functions rather than embedding all logic in one method. It described how a shared mutable dictionary (dialog_state) can be passed by reference through both the orchestrator and the slot-filler without copying, keeping both functions readable. It also clarified that placing the safety check before the slot-filling call inside the orchestrator is the correct pattern so that emergency escalation can short-circuit the slot-filling path entirely.

Value I Added: I designed all architectural decisions before this session: the 8-field dialog_state schema, the 4 slots and their clinical priority order, the emergency circuit breaker requirement (CTAS before slot-filling on every turn), the temperature values for each GPT call (0.3 for extraction, 0.5 for follow-up generation), and the constraint of not modifying the existing run_assessment() function to preserve backward compatibility. The session clarified the structural pattern to use; I then wrote and organized all functions myself. After writing the code, I ran all 6 test scenarios, identified the edge case bug (non-medical input causing JSON parse failure) through my own testing, and decided independently to fix it at the UI layer with a redirect rather than suppressing the error in the backend.

Prompt Session 5: Folium Map Filtering and Dynamic Dropdown Patterns in Streamlit

Date: March 9, 2026 | **Tool:** Claude Code (VS Code Extension)

Context: I had designed the Facility Finder mockup and verified the facilities.csv data. Before writing the page, I needed to understand two specific patterns I had not implemented before: (1) how to re-render a Folium map with filtered markers when a Streamlit filter changes, since Streamlit reruns the script on every interaction, and (2) how to build a dependent dropdown where the City options update dynamically based on the selected Region without triggering unnecessary page reruns.

My Prompt (summary): *"I'm building a Streamlit page with a Folium map and filter dropdowns. When the user clicks Search, I want to filter a pandas DataFrame and rebuild the map with only the matching markers. Since Streamlit reruns the whole script on each interaction, does rebuilding the folium.Map object inside a conditional block (after the search button is clicked) work correctly with st_folium()? Also, for a dependent dropdown where City options depend on the selected Region, should I filter the cities list before passing it to st.selectbox(), or is there a better pattern?"*

Summary of Response: Claude explained that in Streamlit, rebuilding a folium.Map object on each script rerun is the correct approach since st_folium() renders whatever map object it receives; storing the filtered DataFrame in session_state after the Search button click ensures the map and card sections both use the same filtered data. For the dependent dropdown, it described the pattern of filtering the city list from the DataFrame based on the currently selected region before passing it to st.selectbox(), with an "All Cities" prepended as the default option.

Value I Added: I designed all layout specifications, the 6-section page structure, the colour scheme (blue/green/orange for Hospitals/Walk-In Clinics/Urgent Care), the decision to exclude pharmacies from the platform scope, and the content for all Health Tips and Facility Type explanation cards based on my mockup. Understanding the Streamlit rerun and session_state pattern for filtered map rendering allowed me to structure the page logic correctly when writing it. After building the page, I tested all filter combinations myself, discovered that the initial 24-record CSV produced an empty map for most of the Lower Mainland, and independently decided to rebuild the CSV to 45 records with a deliberate city-level distribution strategy. The colour differentiation between Urgent Care (orange) and Walk-In Clinics (green) was my design decision made after testing, recognizing they serve clinically different needs.